Gravitas policy registry review

What this package is

A local review package with a manager-bound dummy token, a transfer manager, a read-only registry API, a fail-closed consumer-client example and reproducible checks. It distinguishes the existing registry's asset/executor checks from token-level transfer enforcement. The new reference contracts are separate from the existing registry and do not alter it.

Start the sandbox

From the repository root, run one command:

```
docker compose -f mb-sandbox-review/docker-compose.yml up --build
```

The container starts a local chain, deploys the registry, plain GUSD, the dummy gated token and manager, approves the token, registers the manager, runs tests and seven scenarios, generates a real local transaction record, runs verifier mutation tests against that record, then starts the API. It prints addresses, policy version, a local development key and sample requests. Both host ports bind to loopback. No public deployment is made.

API specification: `mb-sandbox-review/reference/api/openapi.yaml` (registry-backed reference API). The earlier file-driven mock specification describes a different service and is superseded for this review.

Three answers

Allowed or refused? The token holder initiates the manager call; the manager checks that the asset is approved and the manager is a registered executor, then transfers the fixed dummy token in one transaction. An unapproved asset, unregistered manager or paused registry is refused. The registry read alone does not move tokens.

Can the transfer check be bypassed or an allowance redirected? The token's immutable manager binding rejects direct `transfer` and delegated `transferFrom`; the manager also requires `msg.sender == from`, so a third party cannot use a holder's manager allowance to select another recipient. The token checks asset approval and pause state on every post-construction balance change. Plain GUSD is intentionally outside the gate. These controls apply only to the reference token and do not change the registry or other assets.

What if the API or registry is unavailable? The API reads live state at one pinned block and, after a two-second RPC timeout and one retry, returns HTTP 503 with `DENY / REGISTRY_UNAVAILABLE`. The client fails closed on unreachable API, timeout, malformed or mismatched response; it never reuses ALLOW and invokes its callback only after a complete current ALLOW. A pause returns `DENY / REGISTRY_PAUSED`. The client does not sign or submit a transaction, and its API read is not atomic with a later transfer; the manager/token enforce independently on-chain.

Glossary

- **Gated token** — A token that checks registry state before a balance-changing operation.
- **Executor** — A contract registered to call the registry's subscription check.
- **Approved asset** — A token whose registry asset flag is true.
- **Policy version** — A counter advanced by each recorded registry rule change.

- **Policy hash** — A commitment to a rule change and the preceding hash.
- **Pinned block** — The chain block number used for all reads in one API decision.
- **Pause** — The registry state that makes guarded checks fail.

2. Run and inspect

The Docker image provides Node 22, Foundry 1.9.0 and Solidity 0.8.24 through the separate reference Foundry configuration. Its build needs network access for pinned binaries and source dependencies. The optional `.devcontainer` route uses the same image and requires the reviewer's own account with a cloud development environment; forwarded ports are private.
A second terminal runs the full verification and returns a pass or fail code:

```
docker compose -f mb-sandbox-review/docker-compose.yml exec sandbox \
  bash /workspace/mb-sandbox-review/reference/scripts/verify-all.sh
```

Startup prints the local registry, GUSD, gated token and manager addresses, policy version, API URL and development API key. The default mnemonic and key are public test values, not secrets. They must never be used on a public chain or reused elsewhere. Host ports 8545 and 8787 are bound to `127.0.0.1`.

Native run without Docker

On Linux, install Git, Bash, Node 22, Foundry 1.9.0 (`forge`, `cast`, `anvil`), Python 3 with `venv`, and `curl`. Clone this exact release with its recursive submodules; `forge-std` is nested inside OpenZeppelin:

```
git clone --recurse-submodules --branch mb-review-v5 \
  https://github.com/AbZe628/gravitas-protocol.git
cd gravitas-protocol
git submodule update --init --recursive
[ -e lib/forge-std ] || ln -s openzeppelin-contracts/lib/forge-std lib/forge-std
python3 -m venv .venv-review
source .venv-review/bin/activate
python -m pip install -r mb-sandbox-review/reference/requirements-verification.txt
export ANVIL_HOST=127.0.0.1 API_HOST=127.0.0.1 ANVIL_PORT=8546 API_PORT=8788
bash mb-sandbox-review/reference/scripts/start-sandbox.sh
```

In a second terminal from the same checkout, activate `.venv-review`, set `ANVIL_HOST=127.0.0.1 ANVIL_PORT=8546 API_PORT=8788`, then run `bash mb-sandbox-review/reference/scripts/verify-all.sh`. The start script runs local builds, contract and API suites, seven scenarios and real-record mutation tests, deploys dummy contracts and starts the API. The second command checks the API while RPC is stopped, verifies HTTP 503 / DENY, restores saved state and checks fresh recovery. These native steps need network access for package and compiler downloads; they do not replace hosted container verification.
The API accepts `X-API-Key`. This example uses the default Docker port 8787; the native route above uses 8788:

```
curl -H 'X-API-Key: local-reference-key-only' \
  http://127.0.0.1:8787/v1/policy
```

The Compose file, Dockerfile and startup script are versioned in the release.

3. What the reference code does

`GatedDemoSukuk` is bound at construction to one manager and permits balance-changing transfer calls only from that manager. Its `_update` hook checks registry asset approval and pause state for manager transfers, mint and burn after construction. The constructor's initial mint is exempt because the token cannot be approved before its address exists. Direct `transfer` and delegated `transferFrom` revert even for an approved asset. The existing registry alone cannot intercept a token that never calls it; plain GUSD remains outside the gate.

`DemoSukukManager` binds exactly one token whose immutable manager and registry getters match; that binding cannot be replaced. `transferSukuk` requires the holder to be `msg.sender`, a nonzero unused request ID and a nonzero recipient, then checks the token's approval and the manager's executor authorization through the registry. The manager uses `transferFrom`; the token's hook checks registry state again. It emits `TransferChecked` with the request ID, token, parties, amount, returned policy version and hash. Any revert rolls back state, token movement and events. The caller-equals-holder condition prevents a third party from redirecting the holder's allowance. The source is GatedDemoSukuk.sol and DemoSukukManager.sol; see the reference tests.

The manager's registry call originates from the manager contract; it checks asset approval and executor status. The registry accepts a subscriber argument but does not inspect it to make an eligibility decision. The API classifies asset state only. `amount`, `channel` and `requestId` are validated/logged caller context, not authoritative evidence and do not change the registry decision. No component identifies an investor, checks recipient eligibility or evaluates instrument terms. Future eligibility would require a separately authenticated provider and issuer-approved integration; none is implemented here.

While paused, the current reference token freezes transfer, mint and burn. That trade-off is tested. A real issuer must choose and test its pause policy: keep a full freeze, or separately govern a narrow emergency exit. This package implements only the full freeze.

Reference API boundary

`GET /health` reports HTTP process liveness; it does not prove that the registry is reachable. `GET /v1/policy` reads policy version/hash, block, chain ID, registry, timestamp and pause state. `POST /v1/verify` accepts an asset address, a decimal amount string and request context; only the live asset flag and pause state decide the result. The amount is validated and logged; it is not an investor-eligibility or instrument-terms check. `compliance-client.mjs` validates ALLOW against the expected chain, registry, request ID and required evidence and invokes its callback only on a fresh valid ALLOW. API and transfer are separate operations and not atomic; the client does not sign or send a transaction.

The reader fetches one latest block and performs all state reads against that block tag. It does not return an old ALLOW on RPC error. All routes require the API key. Requests and outcomes are appended to local JSONL; unavailable reads do not invent policy evidence. The helper has bounded retries/deadline and returns local `UNAVAILABLE` on unreachable HTTP API; no transfer callback runs on non-ALLOW or incomplete/mismatched results. The service, reader and helper are server.mjs, registry.mjs and compliance-client.mjs.

4. Scenarios 01 and 02

01 — Dummy sukuk transfer

The local deployment creates plain GUSD, a dummy gated sukuk and the manager. The registry owner approves the token and registers the manager; the holder approves the manager and initiates the transfer. Tests check the holder and recipient balances, both events, current version/hash and caller authorization.
Run:

```
bash mb-sandbox-review/scripts/run-scenario-1.sh
```

Plain GUSD has no registry-aware transfer hook. The reference token checks its own approval on each balance-moving route. This is a dummy local transfer, not a real subscription or sukuk issuance. See the scenario source.

02 — Approved asset pair

The existing registry view returns true for a pair only when both asset flags are approved. That view does not move tokens or constrain a token that ignores the registry. The runner checks both the existing view and a successful transfer through the gated token and manager.
Run:

```
bash mb-sandbox-review/scripts/run-scenario-2.sh
```

See the scenario source and runner.

What the assertions establish

Case	Setup and action	Observable result
01 — Transfer	Approved gated token, registered manager and holder allowance; request a 100-token transfer.	Holder decreases by 100; recipient increases by 100; manager event matches the current version and registry hash. See captured run 01.
02 — Pair and gate	Query two registry-approved assets, then transfer the approved token through the registered manager.	Pair view returns true for the approved pair; transfer succeeds. The revoked-asset refusal is checked in Scenario 03. See captured run 02.

5. Scenarios 03 and 04

03 — Unapproved asset or executor

An unapproved asset fails with the registry's exact asset-compliance revert. A manager that is not registered as an executor fails with the exact caller-authorization revert. A third party cannot call the registered manager using the holder's allowance. Each path preserves balances, allowance and request-ID state. The registry checks the manager as caller; it does not check subscriber identity or eligibility.
Run:

```
bash mb-sandbox-review/scripts/run-scenario-3.sh
```

See the scenario source.

04 — Rule change and policy history

Revoking the token advances the policy version and changes the hash. The earlier version's hash is unchanged. The new commitment is recomputed from the previous hash, `asset`, token address and `false`. Re-approval advances the version again; it does not roll back history.
Run:

```
bash mb-sandbox-review/scripts/run-scenario-4.sh
```

The manager event refers to the registry version/hash used in the transfer transaction. The commitment identifies an ordered policy change; it does not contain a ruling or prove an instrument's status. See the scenario source.

Expected checks

Change	Expected check
Asset revoked	Exact revert `GPR: Asset not Shariah-compliant`; holder and recipient balances do not change.
Manager not registered	Exact revert `GPR: Calling contract not an authorized executor`; no token movement.
Version N revoked	Version advances to N+1; prior hash remains; new hash uses `keccak256(abi.encode(previousHash, keccak256("asset"), token, false))`.
Asset re-approved	Version advances again; transfer can proceed with the new current version and hash.

The captured outputs are Scenario 03 and Scenario 04.

6. Scenarios 05 and 06

05 — Pause or unavailable node

A pause blocks the manager and direct gated-token transfer. The live API returns `DENY / REGISTRY_PAUSED`. If Anvil stops while the API stays up, `/health` returns 200 but `/v1/verify` returns HTTP 503 with `DENY / REGISTRY_UNAVAILABLE` after its configured timeout and retry. After local state is restored, a fresh live read can return ALLOW. A separately tested unreachable HTTP API returns client `UNAVAILABLE`; its transfer callback is not called.
Run the contract checks:

```
bash mb-sandbox-review/scripts/run-scenario-5.sh
```

The full verifier also pauses the actual local registry, checks the API response, stops the local chain while the HTTP server remains available, checks health plus 503, restores saved state and checks recovery. Client tests independently cover full API outage, timeouts, bounded retries, malformed responses and no callback on non-ALLOW. See the verification script and client tests.

06 — Unauthorized changes and transfer routes

Only the registry owner can change the asset, router and executor lists. The reference token rejects direct and delegated transfers even when approved; only its fixed manager can move it. The holder must be the

manager caller, and an unregistered manager, unapproved asset or paused registry still reverts atomically. Plain GUSD can move because its code never asks the registry.
Run:

```
bash mb-sandbox-review/scripts/run-scenario-6.sh
```

The package does not change the old token or registry. A real issuer would still have to choose the token code, protect approval and pause authority, and test the complete issuance and redemption routes. See the scenario source.

Route comparison

State	Gated demo token	Plain GUSD contrast
Asset approved	Direct transfer and delegated `transferFrom` still revert; a third party cannot redirect a holder's allowance. Only the holder-initiated bound manager route can succeed.	Transfer still succeeds because this token does not call the registry.
Asset not approved	Holder-initiated manager route reverts atomically; balances and request-ID state remain unchanged.	Transfer still succeeds because this token does not call the registry.
Registry paused	Direct and delegated balance changes fail closed with the pause error.	Its transfer is still independent of the paused registry.
Registry restored	The owner unpauses; the approved, registered manager route can proceed again after holder initiation.	No registry recovery is needed for a plain-token transfer.

The captured outputs are Scenario 05 and Scenario 06.

7. Scenario 07 — One-transaction audit record

The manager transfer emits an ERC-20 `Transfer` and a `TransferChecked` event in the same transaction. The included local record is generated by Scenario 07 from a real local Anvil transaction and is re-verified against RPC, receipt and registry history. The verifier requires an expected chain ID, configured deployments and a minimum confirmation count.

Field	Recorded value
Chain and deployments	Private local Anvil chain 31337; deployment addresses are recorded in the generated JSON.
Transfer origin	The token holder's transaction; the signer must equal the `from` address.
Transfer evidence	Manager calldata, receipt status, canonical block hash, ERC-20 `Transfer`, and manager `TransferChecked`, all bound to the same transaction/block.
Policy evidence	Registry state at receipt block; `TransferChecked` version/hash; ordered `PolicyUpdated` history and recomputed hash chain.
Confirmations	Explicit `--confirmations` value; the observed count is stored in the record. Local sandbox defaults to one.

The verifier source validates transaction calldata, holder signer, receipt, block/confirmation binding, both logs, pinned registry getters and ordered policy history. A separate negative test script mutates the real record and local RPC responses to exercise wrong versions/hashes/amounts/parties, missing or cross-transaction events, history gaps/order changes, altered receipts and block hashes. It expects every tamper to fail and runs valid verification under `python -O`.

The older composed example combines separate operations; it is not evidence of an executed transfer. A local record proves only that this dummy code executed on a private test chain; it does not identify an investor, establish eligibility, prove legal or religious status, or show a public-chain transfer.

8. What production work remains

Area	Status
Transfer enforcement	**Reference implementation exists (unaudited):** immutable token-to-manager binding, holder-initiated manager call, `msg.sender == from` check, executor/asset/pause checks, direct/delegated route denial and request-ID replay protection. **Production work remaining:** issuer lifecycle, signature/consent design, other asset routes, issuance/redemption and independent review.
One-transaction record	**Reference implementation exists (unaudited):** receipt, calldata, sender, block/confirmation, both event logs and policy history are rechecked against RPC; mutated real records and RPC replies are expected to fail. **Production work remaining:** approved storage/retention, monitoring, reorg handling, reconciliation and privacy.
Registry-backed HTTP and outages	**Reference implementation exists (unaudited):** pinned-block asset decisions, bounded RPC retry, no cached ALLOW, separate HTTP/RPC outage tests, and a callback-gate client that fails closed. API and later transfer are not atomic. **Production work remaining:** signing/transaction integration, key custody, deployment, monitoring, privacy and operations.

Area	Status
Investor eligibility	**Not built.** The registry checks asset status and manager executor status. Its subscriber parameter is not inspected. No identity, KYC/AML, sanctions, accreditation, suitability or recipient-eligibility decision exists. Caller-supplied amount/context/request ID is not authoritative evidence.
Ruling link	**Not built.** No ruling identifier, source document or approval workflow is connected to registry changes.
Governance changes	Partly built in the core repository. GravitasTimelock is deployed on testnet; ownership of the registry has not been handed to it. The engines are single-step Ownable and there is no separate pauser (docs/KNOWN-ISSUES.md). The reference package adds no governance process.
Independent audit	**Not built.** The reference contracts and API have not received an independent audit.
Funded shared testnet	**Not built.** No shared testnet or funded account was created.
Hosted API with managed keys	**Not built.** The API runs locally and prints a development key.

`checkSubscriptionCompliance(subscriber, token)` accepts a subscriber address but the registry does not inspect it. The API checks asset approval and pause state only. These components do not screen investors. A future identity/eligibility provider must be separately authenticated, governed and integrated; no provider or related API input exists here.

9. Candidate sequence before any production use

Every stage below is **not built**. It is a decision and work sequence, not a deployment commitment.

Stage	Required work
1. Define the instrument and policy	Obtain issuer-approved asset and ruling identifiers, document the rule set and decision authority, and define which transfer, redemption, mint and burn events are in scope. Not built.
2. Set eligibility and governance	Design a separately authenticated identity/eligibility provider and how verified claims are bound to transaction intent; choose approval, pause, revocation and emergency-exit authorities; define review, delay, separation of duties and evidence retention. Caller-provided context is not evidence. Not built.
3. Integrate issuer systems	Adapt and test signed holder consent, token, manager, subscription, redemption and identity-provider flows against the chosen instrument. Bind policy versions to transaction records and reconcile rejected operations. Not built.
4. Operate the API and records	Select hosting, key custody, access control, privacy and retention rules, monitoring, alerting, backups and recovery tests. Define what happens if the registry, RPC, API or audit sink is unavailable. Not built.
5. Independent security review	Complete a threat model, dependency review, independent contract and service audit, remediate findings, and repeat tests on the actual deployment configuration. Not built.
6. Staged deployment and approval	First obtain issuer sign-off for a public test environment and funding, then validate operations and monitoring. Mainnet deployment requires separate approvals, verified addresses, key ceremonies, incident procedures and a release decision. None of this has been done.

The sequence must be revised for the issuer's legal, operational and instrument requirements. The local prototype only informs the engineering questions; it does not satisfy any stage above.

10. Verification results

Local results

Check	Result
Existing root Foundry project	`forge build` and tests passed; 86 tests, 0 failed, 0 skipped across 9 suites. The registry and root Foundry config were not modified.
Reference Foundry project	Build passed; 18 passed, 0 failed, 0 skipped; 256 fuzz runs.
Seven scenarios	All seven runners passed on the private local chain; focused outputs are saved in the dated verification evidence.
Reference API and consumer client	19 passed, 0 failed; covers auth, input validation, RPC/API outages, timeout/retry bounds, malformed results and callback gating.
Live API	Registered dummy token ALLOW, unknown token DENY, wrong key 401; real pause DENY then recovery ALLOW.
Actual RPC outage	HTTP API remained up, `/health` stayed 200, while stopped Anvil produced HTTP 503 / `REGISTRY_UNAVAILABLE`; restored state gave a fresh ALLOW.
Real manager record	Generated from Anvil; receipt, calldata, signer, events, block/confirmation binding and ordered policy hash chain reverified against RPC.
Audit-verifier negatives	16 serialized-record mutations and 5 RPC-proof mutations were rejected; valid record still verified under `python -O`.
API specification and dependencies	OpenAPI 3.0.3 validation passed; `npm audit` reported zero vulnerabilities.

The raw command outputs and generated local audit record are included in the dated native verification evidence. These outputs are local-checkout evidence; the hosted workflow checks the exact released commit.

11. Limits and outstanding checks

Hosted Docker verification is delegated to GitHub Actions for the exact v5 release commit. Docker is unavailable in this local sandbox, so the native checks above do not claim to verify the container. At the time this PDF was built, the latest successful hosted run was run 42820021688, on commit `a20ae490244d72951cf3aa998f43fc25729053f2` (tag `mb-review-v4`); it built and started the Compose sandbox, completed full verification, captured logs and shut down successfully. This is evidence for v4 only. The final v5 commit's Actions result is reported separately with the release details; it is not claimed here before that run exists.

Limits of this review

The reference code is unaudited and is not an issuer product; the dummy sukuk has no legal or religious standing. Local checks prove only their specific outcomes. They do not establish eligibility, a ruling, regulator approval, hosted availability or mainnet readiness. No public deployment was made, and the full-freeze pause policy is not a universal recommendation. The earlier registry review describes the original token bypass; it does not imply that token was changed.